

# Build a Mars Rover, then value it three ways

Create your own *Gymnasium* environment and estimate its value function with DP, Monte Carlo, and TD.

**What you'll do.** Stop reading about MDPs and build one. You'll create a small *Mars Rover* world as a real gymnasium environment, then compute its value function three ways: once on paper-logic with the model (DP / value iteration), and twice purely from simulated driving (Monte Carlo and TD). Submit one notebook (.ipynb) + a half-page write-up. **[100 pts]**

**Use Gymnasium, not Gym.** The original `openai/gym` is deprecated and unmaintained; the living standard is **Gymnasium**, maintained by the **Farama Foundation** (same API, drop-in successor). Everything below is written for Gymnasium  $\geq 1.0$  (tested on 1.3).

## Setup – the modern stack [5]

DO THIS FIRST

```
# 1. fresh environment (use uv if you have it: `uv venv` then `uv pip install ...`)
python -m venv .venv && source .venv/bin/activate
pip install gymnasium numpy matplotlib

# 2. sanity check
python -c "import gymnasium as gym; print('gymnasium', gym.__version__)" # >= 1.0
```

The whole Gymnasium API you need fits in six lines. Note the **5-tuple** returned by `step` (this is the modern API; old Gym returned 4):

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", is_slippery=True) # a built-in tabular env
obs, info = env.reset(seed=0) # reset() -> (observation, info)
obs, reward, terminated, truncated, info = env.step(env.action_space.sample())
# step() -> (observation, reward, terminated, truncated, info). Episode ends if terminated OR truncated.
# Tabular envs expose the MODEL: env.unwrapped.P[state][action] = [(prob, next_state, reward, done), ...]
```

**Submit:** the printed Gymnasium version and the 5-tuple from one `step()` on FrozenLake.

## Problem 1 – Build the Mars Rover environment [25]

CREATE A CUSTOM GYMNASIUM.ENV

A  $5 \times 5$  dusty crater field. The rover starts at  $(0,0)$ , must reach the science target  $(4,4)$  (reward  $+10$ , episode ends), and must avoid two craters at  $(2,2)$  and  $(1,3)$  (reward  $-10$ , episode ends). Every move costs  $-1$ . The terrain is dusty: the rover goes where it intends with prob **0.8**, and **slips** to each perpendicular direction with prob **0.1**.  $\gamma = 0.95$ . States are the 25 cells ( $s = \text{row} \times 5 + \text{col}$ ); actions are  $0=\text{up}, 1=\text{right}, 2=\text{down}, 3=\text{left}$ .

Here is the skeleton. Fill the two TODOs, then it runs.

```
import numpy as np, gymnasium as gym
from gymnasium import spaces

GRID=5; GOAL=(4,4); CRATERS={(2,2),(1,3)}; SLIP=0.1; GAMMA=0.95
MOVES={0:(-1,0),1:(0,1),2:(1,0),3:(0,-1)}; PERP={0:[3,1],1:[0,2],2:[1,3],3:[2,0]}
def rc(s): return divmod(s,GRID)
def idx(r,c): return r*GRID+c

class MarsRoverEnv(gym.Env):
    def __init__(self):
        super().__init__()
        self.observation_space = spaces.Discrete(GRID*GRID)
        self.action_space = spaces.Discrete(4)
        self.goal = idx(*GOAL); self.craters = {idx(*c) for c in CRATERS}
        self.terminals = self.craters | {self.goal}
        self.P = self._build_model()

    def _move(self, s, a):
        # move one cell, clipped at the walls
        r,c = rc(s); dr,dc = MOVES[a]
        return idx(min(max(r+dr,0),GRID-1), min(max(c+dc,0),GRID-1))

    def _reward(self, s_next):
        # TODO 1: return +10 at the goal, -10 in a crater, -1 otherwise
        ...

    def _build_model(self):
        # P[s][a] = [(prob, s_next, reward, terminated), ...]
        P = {s:{a:[] for a in range(4)} for s in range(GRID*GRID)}
        for s in range(GRID*GRID):
            for a in range(4):
                if s in self.terminals:
                    P[s][a]=[(1.0,s,0.0,True)]; continue
                outcomes={}
                # TODO 2: intended action gets prob (1-2*SLIP); each PERP[a] slip gets prob SLIP.
                #          accumulate into outcomes[s_next] += prob (use self._move)
                ...
                P[s][a]=[(p,sn,self._reward(sn),sn in self.terminals) for sn,p in outcomes.items()]
        return P

    def reset(self, seed=None, options=None):
        super().reset(seed=seed); self.s = idx(0,0)
        return self.s, {}

    def step(self, action):
        tr = self.P[self.s][action]
        i = self.np_random.choice(len(tr), p=[t[0] for t in tr])
        _, s_next, reward, terminated = tr[i]
        self.s = s_next
        return s_next, reward, terminated, False, {}
```

1. Complete TODO 1 and TODO 2. **[10]**
2. Register and run it with a *random* policy for one episode; print each  $(s, a, \text{reward}, s_{\text{next}})$ . Confirm `step()` returns the 5-tuple and the episode ends at a crater or the goal. **[8]**
3. Why does the env build `self.P` (the full transition model)? Which of the three algorithms will be allowed to read it, and which two are *forbidden* from touching it? **[7]**

**Submit:** your completed MarsRoverEnv, the printed random episode, and the one-paragraph answer.

## Problem 2 – DP: value iteration (uses the model) [25]

### DYNAMIC PROGRAMMING

Because you built the dynamics, you have the model `env.P` — so you can compute the value without ever calling `step()`. Implement value iteration:

$$V(s) \leftarrow \max_a \sum_{s'} P(s' | s, a) [r + \gamma V(s')] \quad \text{until it stops moving.}$$

```
def value_iteration(env, gamma=0.95, theta=1e-6):
    import numpy as np
    V = np.zeros(env.observation_space.n)
    while True:
        delta = 0
        for s in range(env.observation_space.n):
            if s in env.terminals: continue
            # TODO: q_a = sum over (p, sn, r, term) in env.P[s][a] of p*(r + gamma*V[sn]*(not term))
            #       V_new = max over a of q_a ; track delta = max(delta, |V_new - V[s]|) ; V[s] = V_new
            ...
        if delta < theta: break
    # then extract the greedy policy: policy[s] = argmax_a q_a
    return V, policy
```

1. Implement `value_iteration`. Print  $V^*$  as a  $5 \times 5$  grid (`v.reshape(5,5).round(2)`). [12]
2. Print the greedy policy as arrows (`^ > v <`). Does it sensibly steer toward (4,4) and around the craters? [8]
3. Change SLIP from 0.1 to 0 (perfect terrain) and re-run. What changes in the values near the craters, and why? [5]

**Submit:** the value grid, the arrow policy, and your slip-vs-no-slip observation.

## Problem 3 – MC and TD: learn it by driving [30]

### MODEL-FREE – ONLY `ENV.STEP()`

Now estimate the value of the greedy policy from Problem 2 **without the model** — your code may call `env.reset()` and `env.step()` only. Implement both updates:

$$\text{MC: } V(s) \leftarrow V(s) + \alpha [G - V(s)] \qquad \text{TD(0): } V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$$

1. **Monte Carlo.** Run many episodes under the greedy policy; for each, compute the return  $G$  backward and update every visited state. Report the value grid after 1k, 5k, 20k episodes. [10]
2. **TD(0).** Update every step using  $r + \gamma V(s')$ . Report the value grid at the same checkpoints. (Remember: at a terminal next-state, the bootstrap term is 0.) [10]
3. **Compare.** Pick the start state  $(0,0)$ . Plot  $V(\text{start})$  vs episodes for MC and TD on one chart, with the DP value as a dashed line. Which is noisier? Which settles faster? [6]

4. **Explain.** In one short paragraph, tie the picture to *what each method's target is* (full return vs one step + a bootstrap). [4]

**Submit:** both functions, the three value grids each, the convergence plot, and the paragraph.

## Problem 4 – Build your own world [10]

---

CREATE A SECOND GYMNASIUM.ENV

Invent a different small environment as a `gymnasium.Env` with at least one stochastic transition — e.g. a **delivery drone** dodging wind gusts, an **elevator** serving random floor calls, or a **warehouse robot** on a slippery floor. Run *at least one* of DP / MC / TD on it and show the value function. Keep it small enough to print.

**Submit:** your new env class, what S/A/P/R/ $\gamma$  mean in it, and one value-function result.

## Problem 5 – Short answers [5]

---

1. DP read `env.P`; MC and TD only called `env.step()`. In a real Mars rover, which of these is realistic, and why? [2]
2. You hit a crater on episode 1. With MC, when does the state *before* it learn it was bad? With TD? [2]
3. Everything here found the value of a *fixed* policy. What one idea turns this into finding the *best* policy from experience — and which lecture is that? [1]

**Looking ahead (optional, for the curious).** Classical envs like this live in **Gymnasium**. When you move to *agent*ic RL — LLMs using tools over many turns — the ecosystem changes: frameworks like **OpenEnv** (Meta), **Verifiers** (Prime Intellect), **NeMo Gym** (NVIDIA), **SkyRL** and **GEM** run thousands of environments in parallel. See the HuggingFace guide “*The ultimate guide to RL environments*” (Kolavi et al., 2026) for that landscape. Same Bellman backup underneath — just a much bigger world.